

Serial Communication System Documentation

TDV 2215 Terminal - Z80SIO Serial Transmission Analysis

Introduction

The TDV 2215 terminal implements a sophisticated serial communication system using the Z80SIO (Serial Input/Output) controller. The system employs a state-machine driven approach to manage data transmission between the terminal and host computer, ensuring reliable character-by-character communication with proper flow control and error handling.

Send Logic Overview

The transmission logic operates through a coordinated state machine that manages four distinct phases: IDLE (waiting), READY_TO_SEND (prepared), SENDING_DATA (active transmission), and ERROR_OR_DISABLED (fault condition). When a character needs to be transmitted, the system first queues it in a circular buffer, transitions through the appropriate states based on hardware availability and system mode, and uses interrupt-driven completion handling to maintain continuous data flow without blocking the main terminal operations.

Memory Layout & Function Mapping

Core Functions

Function Name	Address	Original Name	Purpose
<code>terminal_character_output</code>	0x2557	<i>(unchanged)</i>	Main character output handler
<code>serial_start_transmission_if_ready</code>	0x0ed4	<code>serial_conditional_transmission_start</code>	Conditional transmission initiation
<code>serial_tx_complete_interrupt_handler</code>	0x0e0c	<code>serial_transmission_complete_handler</code>	Transmission completion interrupt handler
<code>enqueue_char_to_keyboard_buffer</code>	0x10d9	<code>keyboard_buffer_operation_1</code>	Add character to transmission buffer
<code>dequeue_char_from_keyboard_buffer</code>	0x10e5	<code>keyboard_buffer_operation_2</code>	Remove character from transmission buffer
<code>write_to_keyboard_tx_buffer</code>	0x1107	<i>(unchanged)</i>	Low-level buffer operation

Supporting Functions

Function Name	Address	Purpose
<code>escape_sequence_parser</code>	<i>(referenced)</i>	Processes ANSI/VT escape sequences
<code>write_character_with_cursor_wrap</code>	<i>(referenced)</i>	Handles cursor positioning and line wrapping
<code>store_byte_in_circular_buffer</code>	<i>(referenced)</i>	Generic circular buffer storage
<code>read_char_from_circular_buffer</code>	<i>(referenced)</i>	Generic circular buffer retrieval
<code>disableMaskableInterrupts</code>	<i>(referenced)</i>	Z80 interrupt control
<code>enableMaskableInterrupts</code>	<i>(referenced)</i>	Z80 interrupt control

Memory Variables

Variable Name	Address	Original Name	Type	Purpose
<code>serial_transmission_state</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	State machine controller (0-3)
<code>system_mode_state</code>	0x5df8	<code>BYTE_ram_5df8</code>	<code>byte</code>	System operation mode
<code>transmission_buffer_empty_flag</code>	0x5e24	<code>some_variable_5e24</code>	<code>byte</code>	Buffer status indicator
<code>keyboard_data_available_flag</code>	0x5f10	<code>BYTE_ram_5f10</code>	<code>byte</code>	Data availability flag
<code>current_output_character</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Character being processed
<code>serial_transmission_busy_flag</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Hardware busy indicator
<code>serial_transmission_config</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Configuration register
<code>cursor_wrap_enabled_flag</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Cursor wrapping setting
<code>z80sio_channel_a_test_status</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Z80SIO status register
<code>keyboard_buffer_counter</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Buffer occupancy counter
<code>keyboard_tx_buffer_write_pointer</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte</code>	Buffer write position
<code>keyboard_tx_buffer_start</code>	<i>(global)</i>	<i>(unchanged)</i>	<code>byte[]</code>	Circular buffer storage

State Machine Architecture

Serial Transmission States

csharp

```
/// <summary>
/// Serial transmission state machine for Z80SIO communication
/// Controls the flow of data between terminal and host system
/// </summary>
public enum SerialTransmissionState : byte
{
    /// <summary>
    /// IDLE - No transmission in progress, ready to accept new data
    /// System is waiting for data to transmit
    /// Z80SIO channel is available and ready
    /// </summary>
    IDLE = 0,

    /// <summary>
    /// READY_TO_SEND - Data is queued and ready for transmission
    /// Transmission hardware is available and initialized
    /// Waiting for hardware to begin transmission
    /// </summary>
    READY_TO_SEND = 1,

    /// <summary>
    /// SENDING_DATA - Transmission is actively in progress
    /// Data is being sent through the Z80SIO channel
    /// Hardware transmission interrupt will signal completion
    /// System should not queue additional data during this state
    /// </summary>
    SENDING_DATA = 2,

    /// <summary>
    /// ERROR_OR_DISABLED - Transmission error or system disabled
    /// Used when communication fails or system is in error state
    /// Requires manual intervention or reset to recover
    /// No transmission attempts should be made in this state
    /// </summary>
    ERROR_OR_DISABLED = 3
}
```

System Operation Modes

csharp

```
/// <summary>
/// System operation mode for terminal functionality
/// Controls how characters are processed and transmitted
/// </summary>
public enum SystemModeState : byte
{
    /// <summary>
    /// NORMAL_OPERATION - Standard terminal operation mode
    /// Normal character processing and transmission
    /// Full escape sequence processing enabled
    /// </summary>
    NORMAL_OPERATION = 0,

    /// <summary>
    /// DIRECT_MODE - Direct transmission mode
    /// Bypasses some processing steps
    /// Immediate transmission when possible
    /// </summary>
    DIRECT_MODE = 1,

    /// <summary>
    /// BUFFERED_MODE - Buffered transmission mode
    /// Uses keyboard buffer for data management
    /// Enables flow control and batching
    /// </summary>
    BUFFERED_MODE = 2
}
```

State Machine Diagram

mermaid

stateDiagram-v2

[*] --> IDLE : System Initialization

IDLE : 0 - IDLE

IDLE : No transmission in progress

IDLE : Ready to accept new data

IDLE : Z80SIO channel available

READY_TO_SEND : 1 - READY_TO_SEND

READY_TO_SEND : Data queued for transmission

READY_TO_SEND : Hardware initialized

READY_TO_SEND : Waiting for transmission start

SENDING_DATA : 2 - SENDING_DATA

SENDING_DATA : Active transmission in progress

SENDING_DATA : Data flowing through Z80SIO

SENDING_DATA : Hardware busy

ERROR_OR_DISABLED : 3 - ERROR_OR_DISABLED

ERROR_OR_DISABLED : Communication error state

ERROR_OR_DISABLED : System disabled

ERROR_OR_DISABLED : Requires manual intervention

IDLE --> READY_TO_SEND : serial_start_transmission_if_ready()\nData available & system_mode

READY_TO_SEND --> SENDING_DATA : Hardware starts transmission\nZ80SIO begins data transfer

SENDING_DATA --> IDLE : serial_tx_complete_interrupt_handler()\nTransmission complete & buf

SENDING_DATA --> READY_TO_SEND : serial_tx_complete_interrupt_handler()\nTransmission compl

IDLE --> ERROR_OR_DISABLED : Hardware failure\nCommunication error

READY_TO_SEND --> ERROR_OR_DISABLED : Initialization failure\nTimeout error

SENDING_DATA --> ERROR_OR_DISABLED : Transmission error\nHardware malfunction

ERROR_OR_DISABLED --> IDLE : Manual reset\nError recovery\nSystem restart

note right of IDLE

Entry Conditions:

- System boot/reset
- Successful transmission completion
- Error recovery

end note

note right of READY_TO_SEND

Entry Conditions:

- serial_transmission_state = 1 in code
- keyboard_data_available_flag != 0
- system_mode_state allows transmission

end note

note right of SENDING_DATA

Entry Conditions:

- Z80SIO hardware starts transmission
- Previous state was READY_TO_SEND
- Data loaded into transmission buffer

end note

note right of ERROR_OR_DISABLED

Entry Conditions:

- Hardware communication failure
- Buffer overflow
- System disabled by operator
- Unrecoverable transmission error

end note

Detailed Function Analysis

terminal_character_output (0x2557)

Purpose: Main entry point for character output processing

Parameters:

- **character_to_output** (byte): Character to be transmitted

Return: **byte** - The processed character

Local Variables:

- **processed_character** (byte): Character after processing
- **should_continue_processing** (undefined1): Loop control flag

Flow:

1. Store character in global **current_output_character**
2. Handle cursor wrapping if enabled
3. Wait for serial transmission to be available
4. Process escape sequences
5. Attempt to start transmission
6. Set transmission busy flag

Key Logic:

c

```
do {
    should_continue_processing = 0;
    processed_character = current_output_character;
} while (serial_transmission_busy_flag != 0);

do {
    escape_sequence_parser(processed_character);
    processed_character = serial_start_transmission_if_ready();
} while ((bool)should_continue_processing);
```

serial_start_transmission_if_ready (0x0ed4)

Purpose: Conditionally initiates serial transmission based on system state

Return: undefined1 - Status/character value

Local Variables:

- in_AF (undefined2): Z80 AF register pair
- uVar1 (undefined1): Accumulator value
- uVar2 (undefined1): Working variable
- bVar3 (bool): Condition flag

State Logic:

```
c

if (system_mode_state != 1) {
    if ((system_mode_state != 2) || (serial_transmission_state == 3))
        goto LAB_ram_0f11;

    if (serial_transmission_state == 0) {
        serial_transmission_state = 1; // IDLE → READY_TO_SEND
    } else {
        enqueue_char_to_keyboard_buffer();
    }

    if (keyboard_data_available_flag == 0)
        goto LAB_ram_0f11;
}
```

Critical Sections:

- Interrupts disabled during state transitions

- Atomic read-modify-write operations

`serial_tx_complete_interrupt_handler` (0x0e0c)

Purpose: Handles transmission completion interrupts from Z80SIO

Return: `undefined1` - Interrupt acknowledgment

Local Variables:

- `in_AF` (`undefined2`): Z80 AF register pair
- `cVar1` (char): Buffer status

State Transitions:

```
c
if ((serial_transmission_state != 3) && (serial_transmission_state != 0)) {
    if (serial_transmission_state == 1) {
        z80sio_channel_a_test_status = 0; // READY_TO_SEND handling
    } else {
        // SENDING_DATA → IDLE or READY_TO_SEND
        if ((transmission_buffer_empty_flag == 0) &&
            (dequeue_char_from_keyboard_buffer(), cVar1 != '\0')) {
            serial_transmission_state = 0; // → IDLE
        } else {
            transmission_buffer_empty_flag = 0;
            serial_transmission_state = 1; // → READY_TO_SEND
        }
    }
}
```

`write_to_keyboard_tx_buffer` (0x1107)

Purpose: Adds character to circular transmission buffer

Parameters:

- Implicit: Character value from Z80 accumulator

Return: `undefined1` - Character value

Buffer Management:

c

```
if (keyboard_buffer_counter != 0xff) {
    keyboard_buffer_counter = keyboard_buffer_counter + 1;
    uVar1 = (ushort)keyboard_tx_buffer_write_pointer;

    if (keyboard_tx_buffer_write_pointer == 0xff) {
        keyboard_tx_buffer_write_pointer = 0xff; // Overflow protection
    }
    keyboard_tx_buffer_write_pointer = keyboard_tx_buffer_write_pointer + 1;
    (&keyboard_tx_buffer_start)[uVar1] = character_value;
}
```

Features:

- Circular buffer with overflow protection
 - Maximum capacity: 255 characters
 - Write pointer wraparound handling
 - Counter-based occupancy tracking
-

Buffer Architecture

Circular Buffer Implementation

The system uses a circular buffer design for managing transmission data:

- **Buffer Size:** 256 bytes (0x00-0xFF)
- **Overflow Handling:** Counter saturation at 0xFF
- **Thread Safety:** Interrupt-protected operations
- **Flow Control:** Buffer full detection prevents data loss

Memory Management

1. **Write Operations:** Protected by interrupt disable/enable
 2. **Read Operations:** Atomic character extraction
 3. **Pointer Management:** Automatic wraparound with overflow detection
 4. **Status Tracking:** Real-time buffer occupancy monitoring
-

Hardware Integration

Z80SIO Interface

The system interfaces directly with the Z80SIO controller:

- **Channel A:** Primary host communication channel
- **Status Registers:** Real-time transmission state monitoring
- **Interrupt-Driven:** Completion signaling through hardware interrupts
- **Flow Control:** Hardware handshaking support

Timing Considerations

- **Interrupt Latency:** Critical sections minimized for responsiveness
 - **State Synchronization:** Atomic transitions prevent race conditions
 - **Buffer Management:** Non-blocking operations maintain real-time performance
-

Error Handling & Recovery

Error Detection

1. **State Validation:** Illegal state transitions prevented
2. **Buffer Overflow:** Saturation prevents memory corruption
3. **Hardware Monitoring:** Z80SIO status register checking
4. **Timeout Protection:** Transmission failure detection

Recovery Mechanisms

1. **Soft Reset:** State machine reset to IDLE
 2. **Buffer Flush:** Clear pending transmission data
 3. **Hardware Reset:** Z80SIO controller reinitialization
 4. **System Restart:** Complete communication subsystem restart
-

Performance Characteristics

- **Throughput:** Limited by Z80SIO baud rate configuration
- **Latency:** Single-character processing with minimal buffering
- **Memory Usage:** 256-byte circular buffer + state variables
- **CPU Overhead:** Interrupt-driven design minimizes polling overhead

This documentation provides a comprehensive view of the TDV 2215 terminal's serial communication system, enabling maintenance, debugging, and potential enhancements to the transmission logic.